

Precio de venta, caja que cuadra y validaciones de captura

Por que confirmar una compra actualiza tambien el precio, por que una caja no se puede abrir dos veces, por que el cierre exige que el dinero cuadre, y que reglas siguen los formularios para no guardar datos imposibles.

1. Precio de venta en la compra. Confirmar actualiza stock, costo Y precio del producto. Es un cambio de regla de negocio, no un detalle: la compra historiza el precio, el producto guarda el vigente.

2. La carrera del stock duplicado. Dos peticiones simultaneas podian confirmar la misma compra y sumar el stock dos veces. Se resolvio con un UPDATE condicional atomico, no con un if.

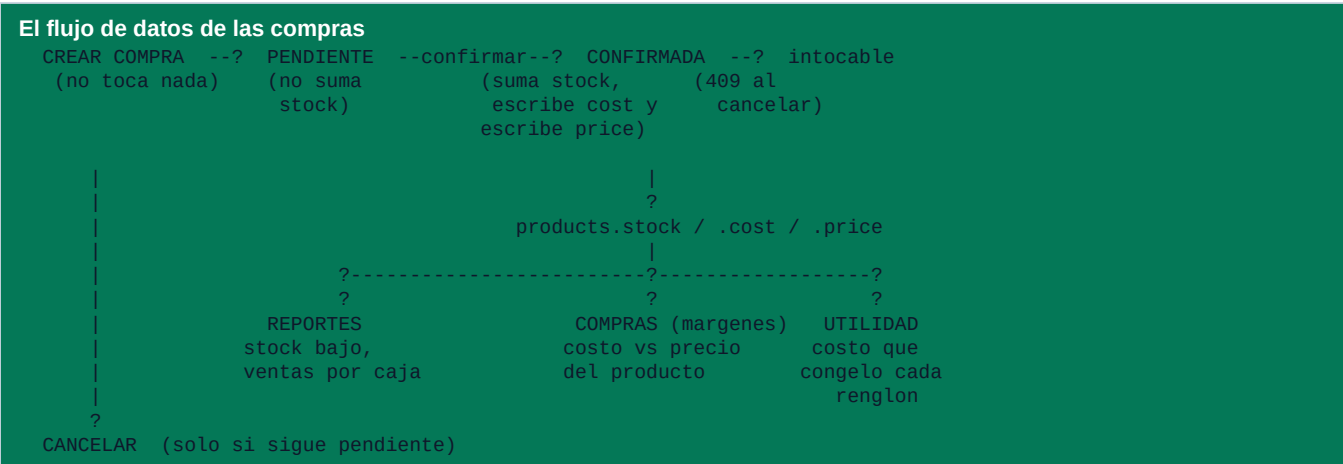
3. Caja: crear, abrir y cuadrar. La caja ahora se registra antes de abrirse, se abre una sola vez, exige un fondo minimo de 100 y no deja cerrar con el dinero descuadrado sin motivo.

4. Validaciones de captura. Stock sin decimales, sin negativos, sin notacion cientifica; SKU con letras y limite; barcode solo digitos; mayusculas en los campos de codigo.

5. Estilos y navegacion. Boton de confirmar destacado, imagenes con marco, seccion activa del menu sombreada, historial mas limpio.

Verificacion. 234 tests en verde y el frontend compilando.

Antes de entrar en detalle conviene ver como se conectan estas funciones, porque no son independientes: casi todas cuelgan de la misma idea, que es **que un datoAutomatico solo vale si un humano lo confirma**.



La flecha de 'confirmar' hacia products es la clave: hasta V3 escribia stock y cost. Con este trabajo escribio tambien price, y eso cambio lo que muestra el reporte de margenes.

Lo que escribe cada columna y quien la lee

Columna	Quien la escribe	De donde sale	Quien la lee
products.stock	CONFIRMAR compra	stock + cantidad del renglón	POS (vender), stock bajo, compras
products.cost	CONFIRMAR compra	unit_cost del renglón	Reporte de margenes, utilidad
products.price	CRUD productos y CONFIRMAR compra (V3)	unit_price del renglón, si viene	POS (cobrar), reporte de margenes
purchase_details.unit_cost	POST /purchases	Lo que paga el proveedor	Historial de la compra (inmutable)
purchase_details.unit_price	POST /purchases	Precio de venta del lote	Historial, y se copia a products al confirmar
sale_details.unit_cost	POST /sales	Costo congelado del producto	Reporte de utilidad (COGS)
cash_registers.number	POST /cash (crear)	Lo escribe el vendedor	Historial, filtro por caja de Reportes
cash_registers.difference_reason	POST /cash/close	Motivo del descuadre	Corte de caja, para poder investigarlo

La tabla explica por que 'unit_price' en la compra y 'unit_cost' en la venta son cosas distintas: la primera historiza una DECISION (a cuanto vender), la segunda congela un HECHO (cuanto costo cuando se vendio).

Este punto cambio una regla que llevaba meses documentada en el codigo: el modulo de compras NUNCA escribia `product.price`, porque se consideraba que el precio de venta es una decision comercial del dueño y que no debía cambiar solo porque el proveedor cobro mas caro.

El flujo real que describio el dueño

La implementacion respeta esa separacion: el renglón de la compra guarda el precio de venta (historizado, no se toca nunca), y al CONFIRMAR se copia a `product.price`, que es el unico lugar donde vive el precio vigente.

Por que nullable y no obligatorio

`unit_price` es nullable, y el significado de NULL es concreto: **esta compra no opina sobre el precio de venta**. Entonces confirmar deja el precio que ya tiene el producto.

- **Compatibilidad.** Las compras ya registradas no tienen el dato, y los clientes que no lo llenen siguen funcionando.
- **Un 0 seria un desastre.** Si fuera NOT NULL con default 0, una compra sin llenar el campo dejaria el producto gratis.
- **Es una decision, no un dato.** Confirma es la accion que aplica cambios, y si no hay opinion sobre el precio, lo correcto es no tocarlo.

El codigo que hace el cambio en la base

`service/impl/PurchaseImpl.java` - `confirm()` escribe stock, cost y price

```
//Ya confirmada (o la confirmó otra petición): 200 idempotente y CERO
//escrituras. No se duplica el stock.
if(filasAfectadas == 0){
    PurchaseEntity yaConfirmada = purchaseRepository.findDetailedById(id)
        .orElseThrow(() -> new PurchaseException("Compra no encontrada", HttpStatus.NOT_FOUND));
    return PurchaseMapper.toDto(yaConfirmada);
}

//Esta petición ganó la carrera: ahora sí se aplica el inventario.
//
//Se refleja el UPDATE sobre la entidad en memoria para devolver un DTO
//correcto. Sin esto habría que volver a leer de la base, y esa segunda
//lectura es un gasto evitable: el repositorio ya-nos dijo que esta
//petición ganó, así que el estado final es conocido sin volver a preguntar.
PurchaseEntity purchase = purchaseRepository.findDetailedById(id)
    .orElseThrow(() -> new PurchaseException("Compra no encontrada", HttpStatus.NOT_FOUND));
purchase.setConfirmed(true);
purchase.setConfirmedAt(cuando);

if(purchase.getDetails() != null){
    for(PurchaseDetailEntity detail : purchase.getDetails()){
        ProductEntity product = productRepository.findById(detail.getProduct().getId())
            .orElseThrow(() -> new PurchaseException(
                "El producto asociado a la compra ya no existe; no se puede confirmar",
                HttpStatus.CONFLICT));
    }
}
```

El comentario largo de mas arriba se movio aqui de forma resumida: lo que queda en el codigo es el 'if (unitPrice != null) setPrice(...)' con la explicacion de por que NULL no se toca. Se considero que el codigo debe explicar la regla sin repetir los tres párrafos de contexto.

Esta es la parte mas tecnica del trabajo, y la que mas conviene entender porque es la clase de bug que no se ve: no da error, no sale en el historial, y un día el inventario simplemente no cuadra.

El problema: idempotencia NO es lo mismo que concurrencia

Habia un `if(purchase.isConfirmed()) return;` y se asumia que eso resolvía el doble clic. Resuelve el reintento **secuencial** (el usuario hace clic dos veces, esperando respuesta entre clics), pero no el **simultaneo**.

Por que el if no alcanza

T1 (peticion A)	T2 (peticion B, al mismo tiempo)
-----	-----
t0 lee confirmed = false	
t1	lee confirmed = false <-- MISMA DATO
t2 suma el stock, escribe y hace commit	
t3	suma el stock, escribe <-- DUPLICADO y hace commit

El problema es que el if lee un valor que era cierto **al momento de leer**, y la lectura y la escritura no son atómicas juntas. Las dos peticiones pasaron el if porque ninguna había escrito todavía.

La solución: un UPDATE condicional que devuelve las filas

En vez de decidir en Java, la decisión se toma en SQL, donde la condición y la escritura se evalúan juntas:

```
repository/PurchaseRepository.java - markConfirmedIfPending()
//¿Cuántas compras tiene un proveedor? (para bloquear su borrado con 409)
long countByProvider_Id(Long providerId);

/**
 * Marca la compra como CONFIRMADA de forma <b>atómica y condicional</b> (V3).
 *
 * <p>Devuelve el número de filas afectadas, y eso es lo importante:
 * <ul>
 *   <li><b>1</b> -> esta petición ganó la carrera: puede aplicar el stock.</li>
 *   <li><b>0</b> -> ya estaba confirmada, así que otra petición se adelantó.
 *     Quien pierde NO debe tocar el stock.</li>
 * </ul>
 *
 * <h3>Por qué esto y no un simple {@code if}</h3>
 *
 * <p>Un {@code if (entity.isConfirmed())} lee la fila y decide en Java. Con
 * dos peticiones simultáneas, <b>las dos leen {@code false}</b> antes de que
 * ninguna escriba, y las dos suman el stock: la compra termina con el doble
 * de mercancía.
 *
 * <p>Esta variante lo resuelve en UNA sentencia de SQL. El {@code WHERE
 * confirmed = false} hace que la segunda transacción espere al lock de fila
 * de la primera y, al reevaluar la condición, ya no se cumpla y devuelva
 * 0. La condición y la escritura son atómicas juntas, que es justo lo que el
 * {@code if} no era.
 *
 * <p>Se prefirió esto a {@code @Lock(PESSIMISTIC_WRITE)} porque no mantiene
 * un lock de fila abierto durante todo el {@code @Transactional} (incluidos
 * los {@code save()} de cada producto del renglón) y porque no depende de que
 * nadie se acuerde de anotar el método de lectura.
 */
@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("""
    UPDATE PurchaseEntity p
    SET p.confirmed = true,
        p.confirmedAt = :when
    WHERE p.id = :id AND p.confirmed = false
""")
```

El número de filas afectadas es lo que manda. Si es 1, esta petición ganó y aplica el stock. Si es 0, otra se adelantó, y esta no toca nada. El `WHERE confirmed = false` hace que la segunda transacción espere el lock de fila de la primera y, al reevaluar, ya no se cumpla.

El servicio, en dos pasos explicitos

service/impl/PurchaseImpl.java - paso 1: ganar la carrera

```

*/
@Override
@Transactional
public PurchaseDTO confirm(Long id){
    //Un solo timestamp para la BD y para la entidad en memoria: si se sacara
    //en dos momentos distintos, el confirmedAt que se guardó y el que ve el
    //usuario podrían diferir en milisegundos y no coincidir con el histórico.
    java.time.LocalDateTime cuando = java.time.LocalDateTime.now();

    // ===== PASO 1: ganar la carrera, de forma atómica (V3) =====
    //
    //Este UPDATE condicional es lo que garantiza que el stock se suma UNA sola
    //vez. Devuelve 1 si esta petición fue la que confirmo, o 0 si otra se
    //adelantó. Es atómico: la condición (confirmed = false) y la escritura
    //(confirmed = true) se evalúan juntas, así que dos peticiones simultáneas no
    //pueden pasar las dos.
    //
    //Va ANTES de tocar stock y NO con un simple if(entity.isConfirmed()) porque
    //ese if lee la fila y decide en Java: con dos peticiones simultáneas las
    //dos leen false antes de que ninguna escriba, y las dos suman el stock.
    int filasAfectadas = purchaseRepository.markConfirmedIfPending(id, cuando);

    //Ya confirmada (o la confirmó otra petición): 200 idempotente y CERO
    //escrituras. No se duplica el stock.
    if(filasAfectadas == 0){
        PurchaseEntity yaConfirmada = purchaseRepository.findDetailedById(id)
            .orElseThrow(() -> new PurchaseException("Compra no encontrada", HttpStatus.NOT_FOUND));
        return PurchaseMapper.toDto(yaConfirmada);
    }
}

```

- **El if no desaparece.** Se *refuerza*: sigue atendiendo el reintento secuencial, y el UPDATE atiende el concurrente. Las dos piezas hacen falta.
- **Se reflejo sobre la entidad en memoria.** Tras el UPDATE, la entidad que se leyó antes ya estaba obsoleta. En vez de releer la base (un query extra), se le pone el estado que el repositorio ya nos dijo que ganamos.
- **Un solo timestamp.** El cuando se calcula una vez y se usa para la BD y para la entidad. Si se sacara en dos momentos distintos, el confirmedAt guardado y el que ve el usuario podrían diferir en milisegundos.

También en las ventas (el mismo hueco desde V2)

SaleImpl.confirm() tenía exactamente el mismo problema desde V2. Se aplicó el mismo patrón. Ahí el dano es menor (no se duplica stock, solo la marca de tiempo), pero se corrigió igual: un confirmed_at distinto del real sería evidencia contable falsa.

```
repository/SaleRepository.java - markConfirmedIfPending()
* caja". Filtra las anuladas igual que el resto de reportes.
*/
@Query("""
    SELECT s
    FROM SaleEntity s
    WHERE s.cashRegister.id = :cashId AND s.cancelled = false
    ORDER BY s.saleDate DESC
    """)
List<SaleEntity> findValidSalesByCashId(@Param("cashId") Long cashId);

/**
 * Marca la venta como CONFIRMADA de forma <b>atómica y condicional</b> (V3).
 *
 * <p>Devuelve el número de filas afectadas: 1 si esta petición ganó la
 * carrera, 0 si otra se adelantó. Mismo mecanismo que
 * {@code PurchaseRepository.markConfirmedIfPending}, y por el mismo motivo:
 * un {@code if (sale.isConfirmed())} lee la fila y decide en Java, así que
 * dos peticiones simultáneas leen {@code false} antes de que ninguna escriba.
 *
 * <p>Aquí el daño de la carrera es menor que en las compras (no se duplica
 * stock, solo se duplica una marca de tiempo), pero se corrige igual: un
 * {@code confirmed_at} distinto del real sería evidencia contable falsa.
 */
@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("""
    UPDATE SaleEntity s
    SET s.confirmed = true,
        s.confirmedAt = :when
    """)
```

El test que blinda la carrera

El test es valioso por un detalle: la entidad que ve la segunda petición tiene `confirmed = false` en memoria, o sea que un `if` la habría dejado sumar el stock. Lo que la salva es que el código decide por las filas afectadas, no por el estado que leyo.

PurchaseImplTest - la carrera de dos peticiones

```
* que la salva es que el código decide por el número de filas afectadas, no
* por el estado que leyó.
*/
@Test
void confirmarCompra_carreraConOtraPeticion_NoSumaStockDosVeces(){
    ProductEntity cafe = product(10L, "Café 1kg", 8);

    PurchaseDetailEntity detail = new PurchaseDetailEntity();
    detail.setProduct(cafe);
    detail.setQuantity(3);
    detail.setUnitCost(new BigDecimal("40"));

    //La compra que ve la SEGUNDA petición: aún marcada como pendiente en
    //memoria, porque la leyó antes de que la primera escribiera.
    PurchaseEntity vistaPorLaSegunda = new PurchaseEntity();
    vistaPorLaSegunda.setId(5L);
    vistaPorLaSegunda.setConfirmed(false);
    vistaPorLaSegunda.setDetails(List.of(detail));

    when(purchaseRepository.findDetailedById(5L)).thenReturn(Optional.of(vistaPorLaSegunda));
    //0 filas = alguien más confirmó primero. ESTE es el valor que manda.
    when(purchaseRepository.markConfirmedIfPending(eq(5L), any())).thenReturn(0);

    PurchaseDTO dto = purchaseService.confirm(5L);

    assertThat(dto.isConfirmed()).isFalse();

    assertThat(cafe.getStock())
        .as("la segunda petición NO debe sumar stock aunque su copia diga pendiente")
        .isEqualTo(8);

    //Ni el producto ni la compra se escriben
    verify(productRepository, never()).save(any(ProductEntity.class));
    verify(productRepository, never()).findById(any());
}

@Test
void confirmarCompra_inexistenteLanza404(){
    when(purchaseRepository.findDetailedById(404L)).thenReturn(Optional.empty());
    when(purchaseRepository.markConfirmedIfPending(eq(404L), any())).thenReturn(0);
}
```

4.1 El modelo cambio: la caja se crea antes de abrirse

Antes el unico punto de entrada era POST /open, que creaba la fila y le ponía el numero en ese momento. El problema es practico: para elegir una caja de una lista, el numero tiene que existir ANTES de abrirla.

Los endpoints nuevos de caja

POST /api/local/cash	CREAR la caja fisica -> {number:"CAJA 1"}, sin abrir (openedAt = null)
GET /api/local/cash/available	Cajas NUNCA abiertas: las candidatas
GET /api/local/cash/next-number	Sugiere "CAJA 2" para prellenar el modal
POST /api/local/cash/open	ABRIR una ya creada -> {number:"CAJA 1", openingAmount:500}

La caja recién creada se distingue de la cerrada por columnas, no por un enum: recién creada tiene opened_at NULL, cerrada tiene closed_at informado. En V1 opened_at ya era nullable, así que no hizo falta alterar la tabla.

Consecuencia de diseño: una caja se abre UNA sola vez

4.2 Abrir: minimo 100 y sin negativos

service/impl/CashRegisterImpl.java - open() valida el fondo

```

    return "CAJA " + (max + 1);
}

// =====
//  ABRIR y CERRAR
//  =====

/**
 * Abre una caja YA REGISTRADA (V3).
 *
 * <p>Tres validaciones, en este orden y por razones distintas:
 * <ol>
 *   <li><b>Solo una caja abierta a la vez</b>: si ya hay una activa, 409.
 *     Es la regla de siempre, no cambia.</li>
 *   <li><b>La caja debe existir</b>: 404 si el número no está registrado. No
 *     se crea al vuelo como antes, porque el número tiene que poder elegirse.</li>
 *   <li><b>La caja no puede haberse usado antes</b>: 409 si {@code openedAt}
 *     ya está informado, porque una caja es un turno.</li>
 * </ol>
 */

```

El minimo de 100 no es solo sentido practico (una caja con 50 no alcanza para una venta): tambien esconde los errores de captura. Escribir 0 casi siempre significa que se olvido el campo, no que el cajon este vacio. Y una caja que abre en 0 contamina el "esperado" del corte sin que nadie lo note.

4.3 Cerrar: el dinero tiene que cuadrar

Este es el punto 5.1 del encargo, y la version elegida fue "bloquear, pero con salida de emergencia". Vale la pena explicar por que las dos mitades hacen falta.

```

service/impl/CashRegisterImpl.java - close() exige el cuadre
        HttpStatus.BAD_REQUEST);
    }
    return openingAmount;
}

/**
 * Cierra la caja exigiendo que el efectivo contado cuadre.
 *
 * <h3>Por qué bloquea y por qué tiene salida</h3>
 *
 * <p>Si el efectivo contado no coincide con el esperado, el cierre se rechaza
 * con <b>409</b>. La razón de fondo: el dinero que falta en el cajón es dinero
 * que salió del negocio, y un corte que "cierra igual" convierte un robo en
 * un descuadre invisible.
 *
 * <p>Pero bloquear sin salida tiene un costo real: si el cajero se equivocó al
 * contar, o un cliente entregó un billete falso y le dio mal el cambio, el
 * turno quedaría encerrado y no se podría cerrar hasta que alguien investigue.
 * Por eso existe {@code differenceReason}: si el cajero está seguro del monto
 * que cuenta, escribe por qué no cuadra y el cierre procede. La diferencia y
 * su motivo quedan guardados juntos, y con eso el descuadre deja de ser
 * invisible.
 */
@Override
@Transactional
public CashResponse close(CloseCashRequest request){
    CashRegisterEntity cash = repository.findByActiveTrue().orElseThrow(() ->
        new CashException("No existe caja abierta", HttpStatus.NOT_FOUND));

    CashSummaryResponse summary = calculateSummary(cash);

    if(request.getClosingAmount() == null){
        throw new CashException(
            "Debes indicar cuánto efectivo hay en la caja", HttpStatus.BAD_REQUEST);
    }
    if(request.getClosingAmount().signum() < 0){
        throw new CashException(
            "El efectivo contado no puede ser negativo", HttpStatus.BAD_REQUEST);
    }
}

```

Las dos mitades

- **Por que bloquear.** El dinero que falta en el cajon es dinero que salio del negocio. Un corte que "cierra igual" convierte un robo en un descuadre invisible: el siguiente turno empieza sin saber que paso.
- **Por que no bloquear en seco.** Si el cajero se equivoco al contar, o un cliente entrego un billete falso y le dio mal el cambio, el turno quedaría encerrado y no se podría cerrar hasta que alguien investigue. El negocio se para por un error de conteo.
- **La salida.** Si el cajero esta seguro del monto que cuenta, escribe el motivo y el cierre procede. La diferencia Y el motivo quedan guardados juntos.
- **Por que el motivo va junto a la diferencia.** "Me sobraron 200" y "me faltaron 200" son problemas opuestos con el mismo sintoma. Un reporte que solo dice "hubo descuadre" no ayuda a resolver ninguno de los dos.

Si el dinero cuadra exactamente, el motivo se ignora y se limpia. Un corte que cerro bien no debe quedar con un motivo de descuadre colgado.

4.4 El contador en vivo (punto 5.5)

cash-facade.ts - efectivoAcumulado y differencePreview

```
/** Cajas nunca abiertas, para el selector. */
availableCash: CashRegister[] = [];

// ===== Modal de CREAR caja (V3) =====
showCreateCashModal = false;

/** Número nuevo, precargado con la sugerencia del backend ("CAJA 7"). */
newCashNumber = '';

// ===== Modal de CERRAR caja =====
showCloseCashModal = false;
closingAmount = 0;

/**
 * Motivo del descuadre (V3). Se muestra SÓLO cuando el dinero no cuadra, y sin
 * él el backend rechaza el cierre con 409.
 */
differenceReason = '';

// ===== Contador en vivo (punto 5.5) =====
/**
 * Efectivo acumulado: fondo inicial + ventas en efectivo, cuando se conoce.
 *
 * <p>Es el número que el cajero quiere ver mientras vende: cuánto hay que
 * should've del cajón ahora mismo. Se devuelve en 0 mientras no haya resumen,
 * para que la plantilla no muestre "null" ni tenga que comprobarlo en cada uso.
 */
get efectivoAcumulado(): number {
  if (!this.cashSummary) return 0;
  return (this.cashSummary.openingAmount || 0) + (this.cashSummary.cashSales || 0);
}
```

El contador muestra **fondo inicial + ventas en efectivo**, que es lo que el cajero quiere ver mientras vende: cuanto debería haber en el cajon ahora. Se implementa como un getter sobre el resumen, así que se recalcula solo en cada deteccion de cambios y no hace falta ningun temporizador.

Cada regla de este bloque viene de un error real de captura. No se inventaron reglas por estetica: se eligieron porque un dato equivocado cuesta dinero o rompe el inventario.

5.1 La idea central: validar el TEXTO, no el numero



5.2 Las reglas y por que cada una

Campo	Regla	Por que
Stock	Entero, no negativo	No existe 1.6 de jabón. Se mide en piezas o kilos, no en 1.6 unidades
Precio	No negativo, máx. 2 decimales	La columna es numeric(38,2): 1.875 se redondearía a 1.88 EN SILENCIO
Ambos	Sin notación científica	1e5 es el atajo para 100000: un error de dedo se vuelve cien mil
Ambos	Sin ceros a la izquierda	000.2 y 007 son errores de dedo. Un 0 solo Sí es válido
SKU	Letras, números y - _ . / , máx. 50	Un SKU no es un número: CHOC-500 es un SKU válido
Barcode	Solo dígitos, máx. 20, TEXTO	Es un identificador: como número perdería los ceros de la izquierda
RFC	Letras y números, máx. 13	Un RFC mexicano tiene 13 (física) o 12 (moral)
Campos de código	Mayúsculas	"choc-500" y "CHOC-500" no deben ser dos SKUs distintos

5.3 Dos capas a proposito



5.4 El codigo del validador del backend

util/InputValidator.java - decimal() con máximo de decimales

```

*/
public static String sku(String valor){
    String codigo = normalizarCodigo(valor);

    if(codigo == null || codigo.isEmpty()){
        return null;
    }

    if(codigo.length() > MAX_SKU_LENGTH){
        throw new IllegalArgumentException(
            "El SKU no puede tener más de " + MAX_SKU_LENGTH + " caracteres (lleva "
            + codigo.length() + ")");
    }

    if(!SKU_PATTERN.matcher(codigo).matches()){
        throw new IllegalArgumentException(
            "El SKU solo admite letras, números y los separadores - _ . / . "
            + "El valor \"" + valor + "\" tiene caracteres no permitidos.");
    }

    return codigo;
}

/**
 * Valida el RFC. Opcional, pero si viene debe tener solo letras y dígitos.
 *
 * <p>Se normaliza a mayúsculas porque un RFC se escribe con letras mayúsculas
 * y compararlo en minúsculas crearía duplicados falsos.
 */
public static String rfc(String valor){
    String codigo = normalizarCodigo(valor);

    if(codigo == null || codigo.isEmpty()){

```

Los métodos lanzan `IllegalArgumentException`, que `GlobalExceptionHandler` traduce a 400. Se prefirió esto a `Bean Validation` porque el mensaje puede decir que límite se rompió y por que, que es más útil que un "validation failed" genérico.

5.5 El filtro de teclado del frontend

validadores.ts - soloDigitos()

```

/** Sanea un pegado en campo de precio: deja dígitos y un solo punto. */
export function sanearPrecio(texto: string): string {
    return texto.replace(/^[^0-9.]/g, '').replace(/(\..*)\./g, '$1');
}

/** Pasa a mayúsculas un campo de código (RFC, SKU, barcode, número de caja). */
export function aMayusculas(event: Event): void {
    const input = event.target as HTMLInputElement;
    const inicio = input.selectionStart;
    const valor = input.value.toUpperCase();
    input.value = valor;
    //Se reposiciona el cursor: si no, al escribir en minúsculas el cursor salta
    //al final y da la impresión de que el campo "se traga" lo que se escribe.
    if (inicio !== null) {
        input.setSelectionRange(inicio, inicio);
    }
}

```

El bloqueo de la 'e' es lo más importante de todo el archivo, y merece una explicación: el `<input type="number">` del navegador ACEPTA 'e' y 'E' y produce valores como 1e5. Ese es el hueco clásico de la validación en el navegador, y por eso se intercepta la tecla antes de que entre al campo.

Se sanean también los PEGADOS con métodos aparte, porque pegar NO dispara `keydown`. Sin eso, el usuario podría pegar "1e5" o "12a" saltándose el filtro de teclas por completo.



Se corrigio al aplicar las reglas del punto 7.2: el SKU ahora acepta letras, numeros y los separadores, pasa a mayusculas y tiene limite de 50. Y el pattern se quito, porque era codigo muerto que sugeria una validacion que no existia.

El barcode no se toco en el mismo sentido: sigue siendo solo digitos, porque es un identificador numerico y perder los ceros de la izquierda rompería el lector.

7.1 El boton Confirmar, destacado a proposito

El boton de confirmar no comparte estilo con el resto de los botones de la tabla: es verde solido, mas alto y con sombra. El motivo es funcional, no estetico: **es la accion de la que depende que el stock exista**. Si el usuario no lo encuentra, la compra se queda pendiente y el inventario nunca suma. Un boton que no se encuentra es peor que un boton que no existe, porque el usuario cree que ya lo hizo.

7.2 La seccion activa del menu

```
sidebar.ts - esActiva()
}

perfil(){
  this.router.navigate(['/perfil']);
}

inicio(){
  this.router.navigate([this.inicioRoute]);
}

/**
 * ¿La ruta dada es la sección donde el usuario está ahora? (PUNTO 6)
 */
```

Se compara la ruta EXACTA y no con includes, porque con includes la ruta "/sales" tambien matchearia "/sales-history" y se marcarían las dos secciones a la vez. Se limpia el query string para que "/reportes?from=2026-01-01" siga marcando Reportes.

7.3 Imagenes con marco

Las imagenes (perfil del usuario y foto de producto) tenian 40x40 sin borde ni fondo, y sobre un fondo oscuro la foto del usuario se perdia. Se agrego borde, sombra y fondo gris. El fondo gris no es decorativo: si la imagen falla en cargar, sale un rectangulo gris y no un icono roto sobre blanco.

7.4 El historial, mas limpio

Se quitaron las columnas "Estado" y "Acciones". El historial es una consulta, no una mesa de trabajo: el cajero mira ventas ya cerradas, y un boton de "Confirmar" ahi no hace falta porque la venta se confirma en el momento del cobro. Las dos columnas ocupaban el espacio de los datos utiles.

El ciclo de vida NO se perdio: la venta anulada sigue identificandose por la fila tachada, y los endpoints PATCH /sales/{id}/confirm y /cancel siguen existiendo en el backend, que es la capa que protege el inventario.

La suite paso de 173 a **234 tests**, todos en verde. Los nuevos no son de cobertura por volumen: cada uno fija una regla que antes podía romperse en silencio.

Clase	Tests	Que fija
PurchaseImplTest	13	Carrera de stock, idempotencia, precio de venta
CashRegisterV3Test	19	Crear, abrir una vez, minimo 100, cuadre y motivo
InputValidatorTest	30	Cada regla de captura, con su caso limite
CashRegisterControllerTest	21	Los endpoints nuevos y sus permisos
PurchaseControllerTest	12	Confirm 200/409/404/403
ReportControllerTest	10	Profit, rango, caja, 404, 403

El caso de prueba que mas documento

InputValidatorTest - no hay 1.6 de jabón

```

    @DisplayName("rechaza más de 20 dígitos")
    void rechazaDemasiadoLargo(){
        assertThatThrownBy(() -> InputValidator.barcode("1".repeat(21)))
            .isInstanceOf(IllegalArgumentException.class)
            .hasMessageContaining("20");
    }
}

// =====
// Enteros: stock y cantidades
// =====
@Nested
@DisplayName("Enteros (stock, cantidad)")
class EnteroTest {

    @Test

```

Y el que más me enseñó: un test que **fallo porque mi expectativa estaba mal**. Escribi que "CHOC/500/1/2/x" debía rechazarse en el SKU, pero ese valor es valido porque / es un separador permitido. El código estaba bien y el test estaba mal. Se corrigió el dato de prueba, no el validador.



Verificacion final: **234/234 tests en verde** en el backend (mvn test -Dtest='!ComprasApplicationTests'), ng build sin errores ni warnings nuevos. El unico test que no corre es ComprasApplicationTests, que es @SpringBootTest y exige una base de datos real.